Program 5: Cyclic Redundancy Check (CRC)

AIM : To write a C program to compute CRC code for the polynomial CRC-12

Description : Cyclic Redundancy Check (CRC) is an error-detection technique used to detect errors in transmitted or stored data. It treats the data as a binary polynomial and performs **modulo-2 division** using a predefined **generator polynomial**. The remainder obtained from the division is called the **CRC** or **Frame Check Sequence (FCS)**, which is appended to the data before transmission.

The **CRC-12** generator polynomial is: $x^{12} + x^{11} + x^3 + x^2 + x + 1$ Its corresponding **binary representation** is: 1100000001111 . CRC-12 provides reliable error detection with only 12 redundant bits. It efficiently detects single-bit, double-bit, and burst errors using simple XOR operations, making it fast, accurate, and suitable for high-speed communication systems.

Algorithm

1. The sender selects a **generator polynomial (G)**.
2. Append **r zeros** to the data, where **r = degree of G**.
3. Divide the modified data by the generator using **modulo-2 (XOR) division**.
4. The remainder is the **CRC**.
5. Append the CRC to the original data and transmit it.
6. The receiver performs the same division.
7. If the remainder is **0**, no error is detected; otherwise, an error is detected.

Program:

```
#include<stdio.h>
#include<string.h>
char data[50],g[20],temp[20];
void XOR()
{
	for(int i=0;i<strlen(g);i++)
	temp[i]=(temp[i]==g[i]?'0':'1');

}
void CRC()
{
	int i,j;
	for(i=0;i<strlen(g);i++)
	temp[i]=data[i];
	do
	{
		if(temp[0]=='1')
			XOR();
		for(j=0;j<strlen(g)-1;j++)
		{
			temp[j]=temp[j+1];
		}
		temp[j]=data[i++];
```

```
        }while(i<=strlen(data));
}
int main()
{
        int flag=0,n,m,k;
        strcpy(g,"110000001111");
        printf("\n Enter data");
                scanf("%s",data);
                n=strlen(data);
                m=strlen(g);
                printf("\n Generate polynomial is:%s",g);
                printf("length of g is %d  length of data is %d\n",m,n);
                for(k=n;k<n+m-1;k++)
                        data[k]='0';

                printf("\n Modified datais %s\n",data);
                CRC();

                printf("\n cyclic redundancy check remainder value us%s",temp);
                int j;
                for(int k=n,j=0;k<n+m-1;k++,j++)
                        data[k]=temp[j];
                printf("\n Modified code word is:%s",data);
        }
```

OUTPUT:

```
Enter data10001
Generate polynomial is:110000001111length of g is 12  length of data is 5
Modified datais 1000100000000000
cyclic redundancy check remainder value us00010101010
Modified code word is:1000100010101010
```

Viva Questions:

1. **What is CRC?**
   Ans: CRC stands for **Cyclic Redundancy Check**. It is an error-detection technique used in computer networks to detect errors in transmitted data.
2. **What is the basic principle of CRC?**
   Ans: The sender performs **modulo-2 division** of the data by a predefined generator polynomial. The remainder is appended to the data and transmitted.
3. **What is the CRC-12 polynomial?**
   A: The CRC-12 polynomial is **x12+x6+x4+x+1**, represented in binary as **1000001010011**.
4. **Other common polynomial are**
   - **CRC-8:** x8+x2+x+1
   - **CRC-12:** x12+x6+x4+x+1
   - **CRC-16:** x16+x15+x2+1
   - **CRC-32:** commonly represented by **0x04C11DB7**
5. **What is Hamming Code?**
   A:Hamming Code is an **error detection and error correction technique** used in data communication.

6. **Who developed Hamming Code?**
   It was developed by **Richard W. Hamming**.
7. **What type of errors can Hamming Code detect?**
   Basic Hamming Code can **detect two-bit errors and correct single-bit errors**.
8. **Where are parity bits placed in Hamming Code?**
   They are placed at positions that are **powers of 2**:
   1, 2, 4, 8, 16, ...
9. **How do you calculate the number of parity bits?**
   If there are **m data bits** and **r parity bits**, then:
   $2^r \geq m + r + 1$
10. **What positions does parity bit P1 check?**
    P1 checks: **1, 3, 5, 7, 9, 11, ...**
11. **What positions does P2 check?**
    P2 checks: **2, 3, 6, 7, 10, 11, ...**
12. **What positions does P4 check?**
    P4 checks: **4, 5, 6, 7, 12, 13, 14, 15, ...**
13. **How is the error position identified?**
    The receiver recalculates the parity bits. The resulting **syndrome** gives the binary position of the erroneous bit.
14. **What is a syndrome?**
    A syndrome is the combination of parity-check results used to **identify the position of an error**.
15. **What does a syndrome of `000` indicate?**
    It indicates **no error** in basic Hamming Code.
16. **If the syndrome is `101`, what does it mean?**
    Binary `101` = decimal **5**, so **bit position 5 contains the error**.